



CSE461

Introduction to Robotics Lab

LAB REPORT - 1

Group: 5

Section: 6

Semester: Summer_2026

Group members :

Alena Halder Tamo	22301046
Tahsin Raihan Robbani	22201344
Mayen Shahriar Kabir	23101107
Mofftasim Hossain Sayem	22299109
Tahmida Tanjim Talukder	22101730

Submitted Date: DD-MM-YYYY

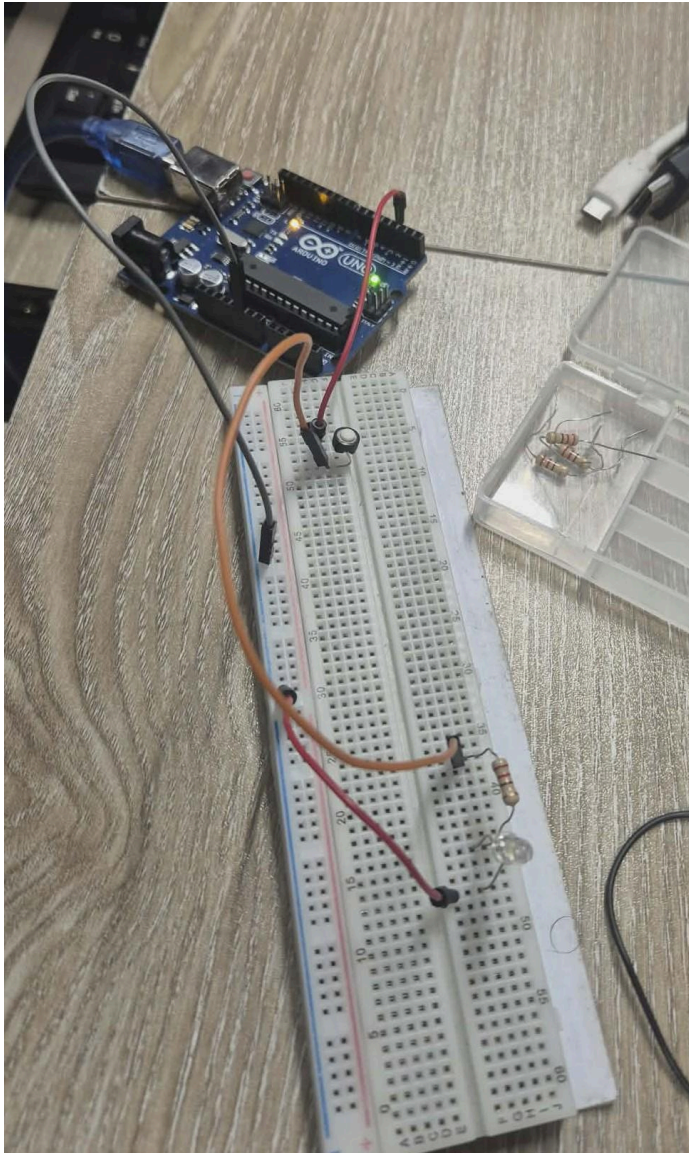
Submitted to -

Pratick Roy Chowdhuruy (Lecturer, BRAC University)

Purba Tahia Hassan (Lecturer, BRAC University)

Lab 1

1. **Equipments:** Arduino Uno R3, LED (Light Emitting Diode), Push-button switch, Resistor (220 Ω for LED), Breadboard, Jumper wires
2. **Experimental Setup:**



3. **Code:**

```
#define BUTTON_PIN 2
```

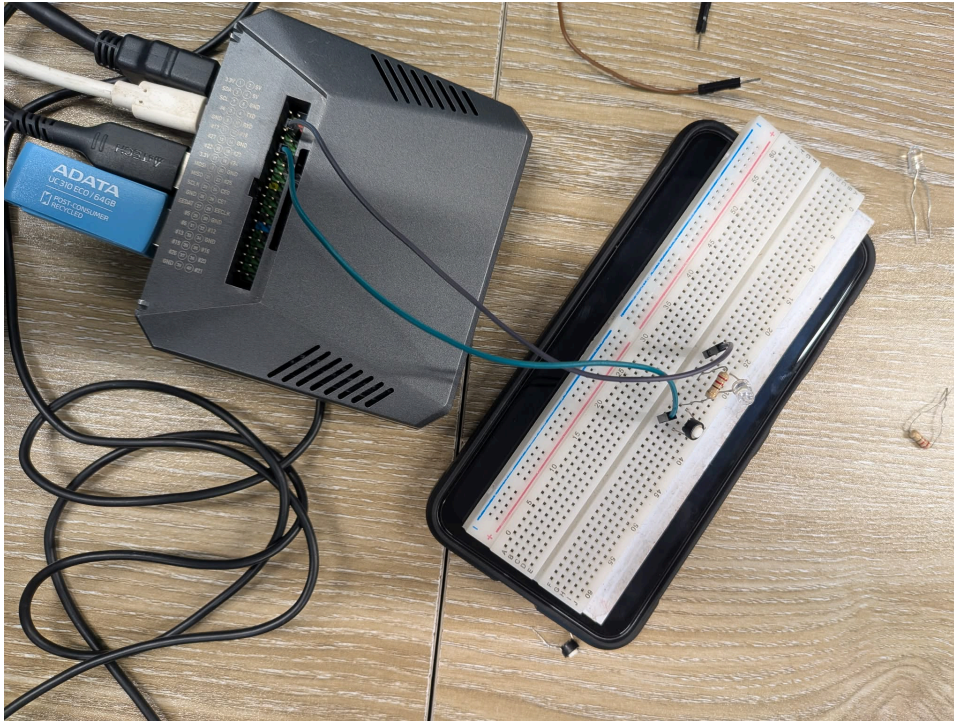
```
#define LED_PIN 8

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("LED ON");
  } else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("LED OFF");
  }
  delay(100);
}
```

Lab 2

1. **Equipments:** Raspberry PI development board, LED (Light Emitting Diode), Resistor (appropriate value for current limiting, typically around 220 ohms), Switch, USB pen drive containing OS
2. **Experimental Setup:**



3. Code:

```
from gpiozero import LED, Button
import time

led = LED(4)

button = Button(14)

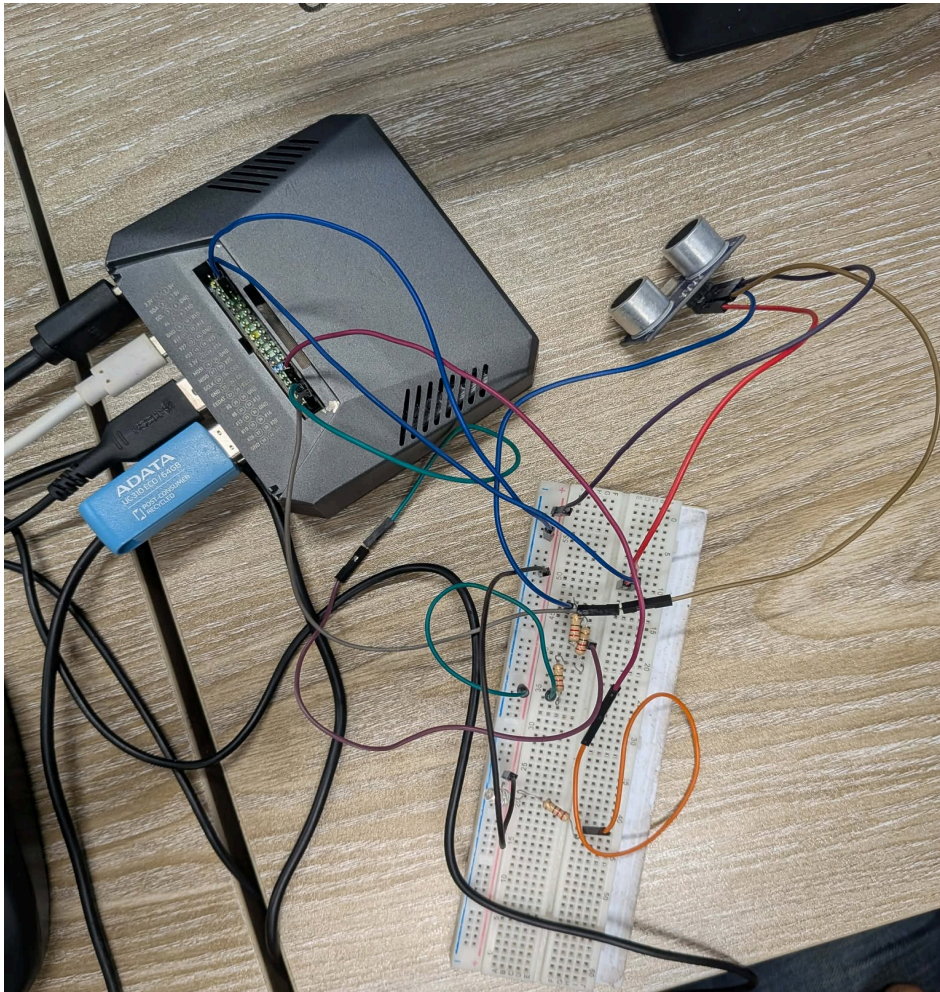
print("Program is running... Press the switch to turn on the LED.")

try:
    while True:
        if button.is_pressed:
            led.on()
        else:
            led.off()

        time.sleep(0.05)
except KeyboardInterrupt:
    led.off()
    print("\nProgram stopped safely. LED turned off.")
```

Lab 3

1. **Equipments:** Raspberry PI development board, LED (Light Emitting Diode), Ultrasonic Sensor (HC-SR04), Resistor (appropriate value for current limiting, typically around 220 ohms), Jumper wires.
2. **Experimental Setup:**



3. **Code:**

```
# Enter Code Here

import RPi.GPIO as GPIO
import time
from gpiozero import LED
```

```
GPIO.setmode(GPIO.BCM)
TRIG = 21 #GPIO21
ECHO = 20 #GPIO20
led=LED(12)

GPIO.setup(TRIG,GPIO.OUT)
GPIO.setup(ECHO,GPIO.IN)

def distance():
    GPIO.output(TRIG, False)
    time.sleep(0.5)
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)
    pulse_start = time.time()
    while GPIO.input(ECHO)==0:
        pulse_start = time.time()
    while GPIO.input(ECHO)==1:
        pulse_end = time.time()
    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * ((343*100)/2)
    distance = round(distance, 2)

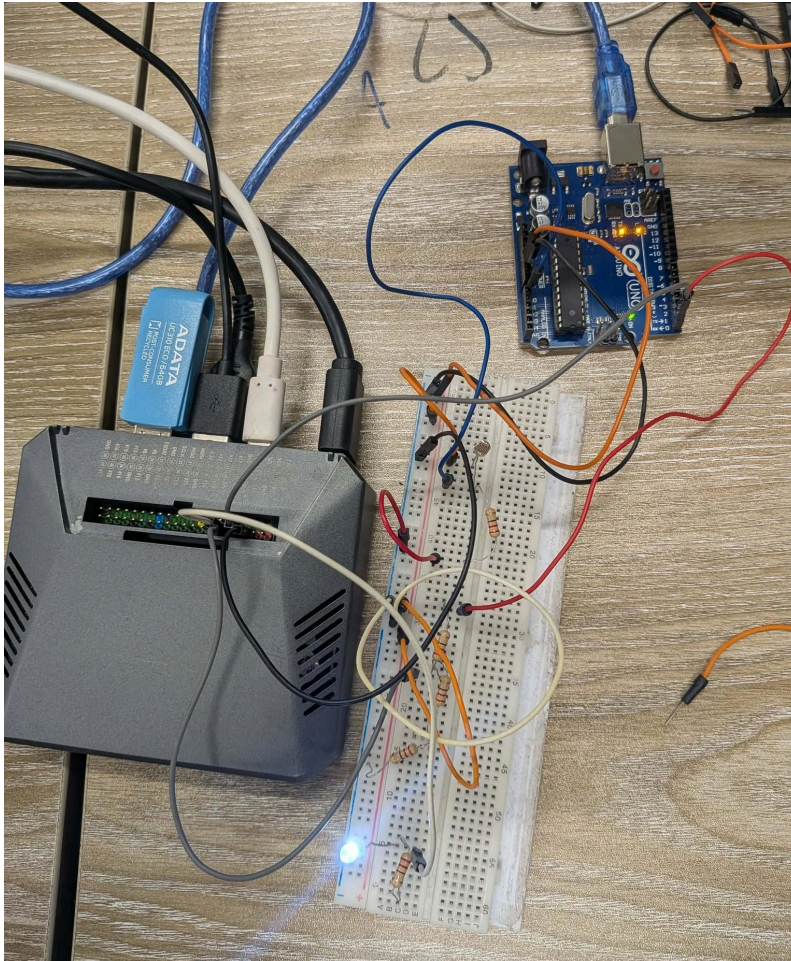
    return distance

while True:
    dist=distance()
    print(dist)
    if dist < 10:
        led.on()
    else:
        led.off()

GPIO.cleanup()
```

Lab 4

1. **Equipments:** Arduino Uno R3, Raspberry PI 4, LED (Light Emitting Diode), LDR sensor, Resistor (220 Ω for LED), Breadboard, Jumper wires
2. **Experimental Setup:**



3. **Code:**

Arduino Code:

```
int ldrPin = A0;
int ldrValue = 0;

void setup() {
  Serial.begin(9600);
}
```

```

void loop() {
  ldrValue = analogRead(ldrPin); // Read LDR value (0–1023)
  Serial.println(ldrValue); // Send value to Raspberry Pi
  delay(100);
}

```

Raspberry PI Code:

```

import serial
import RPi.GPIO as GPIO
import time

# Setup
GPIO.setmode(GPIO.BCM)
led_pin = 18
GPIO.setup(led_pin, GPIO.OUT)

# Initialize PWM
pwm_led = GPIO.PWM(led_pin, 1000) # 1 kHz frequency
pwm_led.start(0)

# Connect to Arduino
arduino = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
time.sleep(2)
norm = 2.5

try:
    while True:
        data = arduino.readline().decode('utf-8').strip()
        if data.isdigit():
            ldr_value = int(data)
            brightness = ((1-(ldr_value / 1023)) ** norm) * 100
            Brightness = max(0, min(brightness, 100))
            pwm_led.ChangeDutyCycle(Brightness)
            print(f"LDR: {ldr_value}, LED Brightness: {brightness:.1f}%")
except KeyboardInterrupt:
    print("Exiting...")
finally:
    pwm_led.stop()
    GPIO.cleanup()
    arduino.close()

```

Questions

1. The HC-SR04's ECHO pin outputs 5V, but the Pi's GPIO can only handle 3.3V. Using one 220Ω resistor on the echo side and two 220Ω resistors (in series) to ground, calculate the voltage that actually reaches the GPIO pin. Show your calculation, and explain in one or two sentences why this arrangement protects the Pi. [Lab 3]

Answer:

Given,

$$V_{in} \text{ (Echo output voltage)} = 5V$$

$$R1 \text{ (resistor on echo/5V side)} = 220 \Omega$$

$$R4 \text{ (two } 220\Omega \text{ resistors in series, to GND)} = R2 + R3 = 220 \Omega + 220 \Omega = 440\Omega$$

Applying the voltage divider formula,

$$V_{out} = V_{in} \times [R1 / (R1 + R4)]$$

$$= 5 \times [220 / (220 + 440)]$$

$$= 3.33V$$

The HC-SR04's ECHO pin outputs a 5 V signal, which is higher than the maximum voltage 3.33 V that a Raspberry Pi GPIO pin can safely handle. The ECHO signal is lowered to ~3.3V before arriving at the GPIO pin with a voltage divider consisting of three 220Ω resistors, in order to keep the Raspberry Pi from being damaged. This is a reduced voltage that is high enough for the Raspberry Pi to interpret as a digital HIGH signal and still within its safe operating parameters. This means that the voltage divider will not allow excessive voltage to reach the GPIO pin and will protect the Raspberry Pi in case of potential damage.

2. The button pin is set to `INPUT_PULLUP` and the code checks for `LOW` when the button is pressed. Explain why pressing the button gives LOW instead of HIGH, and what the internal pull-up resistor keeps the pin at when the button is not pressed. [Lab 1, 2]

Answer:

An `INPUT_PULLUP` pin has an internal pull-up resistor between the pin and the supply voltage (5V on Arduino, 3.3V on Raspberry Pi). When no other resistors are influencing the pin, this resistor pulls the pin towards HIGH. If the button is not pressed then the pin is not

connected to anything else and therefore, the internal pull-up resistor will keep the pin high. When the button is pressed, it directly connects that pin to GND (0V). Because, the weak pull-up resistor is overpowered by GND the voltage on the pin is pulled down to LOW. So, if the button is not pressed, then the internal pull-up resistor will keep the pin HIGH & if the button is pressed - pin is connected to the GND - pin reads LOW. This is why in the code LOW is checked for, in order to know if the button has been pressed or not: When pressed, the LOW signal is generated, not HIGH.

3. In the distance formula, the echo time is divided by 2 before converting to a distance. Explain why that division by 2 is needed, based on how the sensor sends and receives sound. [Lab 3]

Answer:

The reason behind dividing the echo time by 2, is because the HC-SR04 sensor measures the time it takes for the ultrasonic sound wave to hit the object and bounce back to the sensor, which is half the total time. This measured time is the time it takes for the sound wave to move to the object and back, not just the time it takes to get to the object.

On code we can see,

```
pulse_duration = pulse_end - pulse_start  
  
distance = pulse_duration * ((343*100)/2)  
  
distance = round(distance, 2)
```

Measuring time and dividing it by 2 gives the time it takes to travel from the sensor to the object, which the program can use to calculate the correct distance between them.

4. In Experiment 1, the LDR connects between 5V and pin A0, with a 220Ω resistor from A0 to ground. The worksheet mentions that bright light lowers the LDR's resistance. Using that fact, predict whether the number returned by `analogRead(A0)` goes **up or down** when a bright light shines on the LDR, and explain why in one or two sentences. [Lab 4]

Answer:

The resistance of the LDR decreases when bright light shines upon it. The LDR is the upper leg of the voltage divider between the 5V and A0 which means that the higher the

resistance of the LDR the lower the voltage will be across the LDR and the lower the resistance of the LDR the higher the voltage will be across the LDR. This results in more current being available at the A0 output, near the maximum 5V output. Because `analogRead()` converts the voltage at A0 into a digital value between 0 and 1023 (where 0V = 0 and 5V = 1023), a higher voltage at A0 results in a higher `analogRead()` value.

So, when bright light is shone on the LDR, the `analogRead(A0)` value goes UP (increases towards 1023), because the LDR has a lower resistance, which means that more voltage reaches A0, which in turn means that the `analogRead(A0)` value is higher.